Solution: Score of Parentheses

Problem Statement

Given a balanced parenthesis string `s`, return the score of the given string.

Follow the below rules to determine the score of the string `s`.

- `"()"` has score 1.
- `XY` has score `X + Y`, where X and Y both are balanced parentheses strings.
- `(X)` has score `2 * X`, where X is a balanced parentheses string.

Examples

Example 1:

- **Input**: `"((()))"`
- **Expected Output**: 4
- **Justification**: The string represents a pair of parentheses inside another pair, which is again inside another pair. This structure results in a score of `((()))` = `((1*2)*2)` = `4`.

Example 2:

- **Input**: `"(()())"`
- **Expected Output**: 4
- **Justification**: Here, we have a pair of parentheses inside another pair `(()())` and another pair next to it. The total score is `(1*2) + (1*2)` = `4`.

Example 3:

- **Input**: `"(()(()))"`
- **Expected Output**: 6
- **Justification**: The innermost `()` scores 1, its surrounding `()` doubles it to 2, the previous `()` adjacent to the inner pair scores 1 and is then doubled by its surrounding pair, resulting in `2 + (1*2)*2` = `6`.

Solution

To solve this problem, we can use a stack to keep track of the scores at different depths of parentheses. When we encounter an open parenthesis, we push a marker onto the stack to indicate a new depth level. Upon finding a closing parenthesis, we either pop a score off the stack and double it if we're closing a base pair, or we add up the scores at the current depth before doubling. This method works effectively because it directly models the nested structure of the problem, allowing us to calculate the score of deeper levels before adding them to the outer levels. The stack ensures that we're always working on the correct depth of parentheses, making the approach both intuitive and efficient.

This approach is the most effective because it aligns with the problem's inherent hierarchical structure, where scores depend on the depth and arrangement of parentheses. It efficiently processes the string in a single pass, ensuring minimal overhead and straightforward score computation.

Step-by-step Algorithm

1. **Initialize a Stack**: Create an empty stack. Push a 0 onto this stack before starting to iterate through the string. This initial 0 acts as the base score for the level outside all parentheses.

2. **Iterate Through the String**: Loop over each character in the input string `S`. Convert the string into a character array for iteration purposes.

3. **Open Parenthesis (`(`) Handling**:

   - If the current character is an open parenthesis `(`, push a 0 onto the stack. This signifies the start of a new nested level, with a base score of 0 for that level.

4. **Close Parenthesis (`)`) Handling**:

   - When encountering a closing parenthesis `)`, it indicates the end of a current level of nesting. Perform the following steps:
     - Pop the top element of the stack. This represents the score of the immediately nested structure or 0 if it's a direct pair `()`.
     - Calculate the score for the closed structure:
       - If the popped score is 0, this means we just closed a direct pair, so the score is 1.
       - If the popped score is not 0, we have closed a more complex structure, and the score needs to be doubled.
     - Add the calculated score to the next element down in the stack, which represents the score of the next outer level.
     - Push this updated score back onto the stack.

5. **Complete the Calculation**:

   - After processing all characters, the stack's top element will represent the total score of the parentheses string.

Algorithm Walkthrough

Let's consider the input: `"(()(()))"`

1. **Initialization**: Stack is initialized and starts with `[0]`.

2. **First Character -** `(` : Push 0 for a new depth. Stack becomes `[0, 0]`.

3. **Second Character -** `(` : Push 0 for another depth. Stack becomes `[0, 0, 0]`.

4. **Third Character -** `)` : Pop 0 (the score for the innermost `()`), this is a direct pair so score is 1. The stack now has `[0]`. Add 1 to the top element of the stack, resulting in `[0, 1]`.

5. **Fourth Character -** `(` : Indicates a new depth. Push 0. Stack becomes `[0, 1, 0]`.

6. **Fifth Character -** `(` : Another new depth. Push 0. Stack becomes `[0, 1, 0, 0]`.

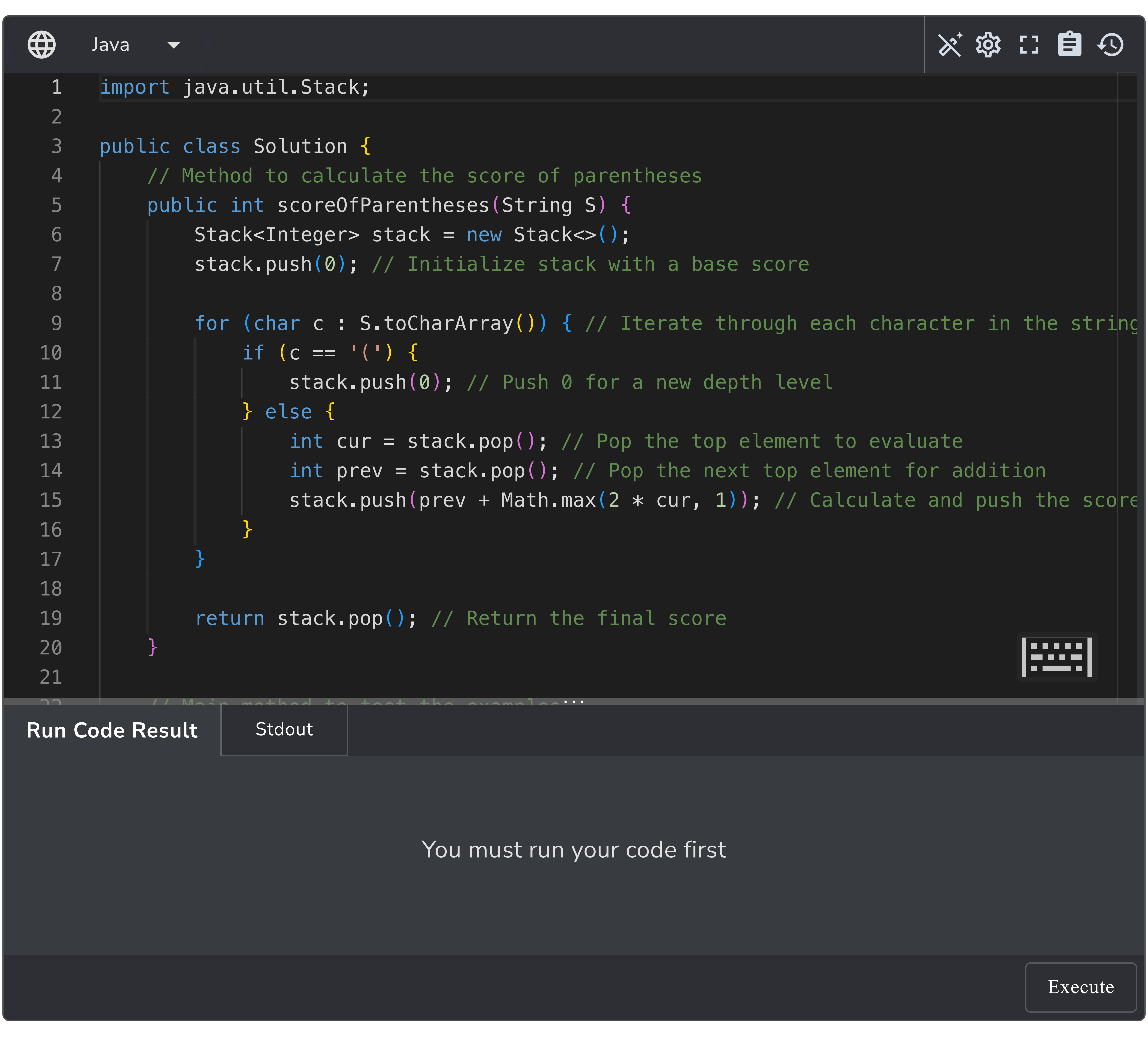
7. **Sixth Character -** `)` : Pop 0, a direct pair, so score is 1. Stack now has `[0, 1, 0]`. Add 1 to the top element of the stack, which results in `[0, 1, 1]`.

8. **Seventh Character -** `)` : Pop 1 (the score for the nested `(()`) since it's not a direct pair, double it (2). The stack now has `[0, 1]`. Add 2 to the top element of the stack, leading to `[0, 3]`.

9. **Eighth Character -** `)` : Pop 3, double it (since it's the score for the nested structure `(()(()))`) to 6. The stack now has `[0]`. Add 6 to the top element of the stack, resulting in `[6]`.

10. **Final Score**: Pop the final value from the stack, which is `6`. This is the total score for the input string `"(()(()))"`.

Code



```java
import java.util.Stack;

public class Solution {
    // Method to calculate the score of parentheses
    public int scoreOfParentheses(String S) {
        Stack<Integer> stack = new Stack<>();
        stack.push(0); // Initialize stack with a base score

        for (char c : S.toCharArray()) { // Iterate through each character in the string
            if (c == '(') {
                stack.push(0); // Push 0 for a new depth level
            } else {
                int cur = stack.pop(); // Pop the top element to evaluate
                int prev = stack.pop(); // Pop the next top element for addition
                stack.push(prev + Math.max(2 * cur, 1)); // Calculate and push the score
            }
        }

        return stack.pop(); // Return the final score
    }

```

You must run your code first

Complexity Analysis

Time Complexity:

The time complexity of the algorithm is $O(n)$, where `n` is the length of the input string `S`. This is because the algorithm involves a single pass through the string, with each character being processed exactly once.

Space Complexity:

The space complexity is also $O(n)$, which is the worst-case scenario where the entire input string consists of opening parentheses followed by closing parentheses, requiring a stack that grows proportionally with the length of the input string.

Next →

← Previous

Score Of Parentheses (Medium)

Maximum Length Of A Concatenated String With Unique Characters (Medium)

☐ Mark as Completed